Utils for Coolmay FX3G PLC

Changelog

V3 01.06.2026

- add - `INCN` and `SHIFT` functions

Description

This library provides useful functions and function blocks.

This a table on data types in this manual and to what types in GX Works2 it refer.

Type in manual	Type in GX Works 2
`INT`	Word[Signed]
`WORD`	Word[Unsigned]/Bit String[16-bit]
`DINT`	Double Word[Signed]
`DWORD`	Double Word[Unsigned]/Bit String[32-bit]
`BOOL`	Bit

ISBON, DISBON

Function to check if a given bit in a WORD or DWORD is on. There is built-in `BON` instruction, but it does not return the value but store it in a parameter you pass to instruction. This is inconvenient. This functions you can use inside expressions.

Variable	Scope	Type	Description
`IN`	INPUT	WORD	The WORD to check. `DWORD` for `DISBON`
`BN`	INPUT	INT	Bit number starts form 0

```
IF ISBON(D100, 2) THEN
    (* The third bit in D100 is ON *)
END_IF;
```

SETB, DSETB

Functions to Sets Bit in a `WORD` or `DWORD` and return modified data.

SETB

Variable	Scope	Type	Description

Variable	Scope	Type	Description
IN	INPUT	WORD	WORD to change
BN	INPUT	INT	What bit to set 0-15

Example:

```
VAR
    wTest : WORD := 2#0000_0000_0000_0000;
END_VAR

wTest := DSETB(wTest, 15);
(* dwTest = 2#1000_0000_0000_0000 *)
```

DSETB

Variable	Scope	Type	Description
IN	INPUT	DWORD	DWORD to change
BN	INPUT	INT	What bit to set 0-31

Example:

```
VAR
    dwTest : DWORD := 2#0000_0000_0000_0000_0000_0000_0000_0000;
END_VAR

dwTest := DSETB(dwTest, 31);
(* dwTest = 2#1000_0000_0000_0000_0000_0000_0000_0000 *)
```

RSTB, DRSTB

Functions to reset Bit in a WORD or DWORD and return modified data.

RSTB

Variable	Scope	Type	Description
IN	INPUT	WORD	WORD to change
BN	INPUT	INT	What bit to reset 0-15

Example:

```
VAR
    wTest : WORD := 2#1000_0000_0000_0000;
```

```
END_VAR

wTest := DRSTB(wTest, 15);
(* dwTest = 2#0000_0000_0000_0000 *)
```

DRSTB

Variable	Scope	Type	Description
IN	INPUT	DWORD	DWORD to change
BN	INPUT	INT	What bit to reset 0-31

Example:

```
VAR
    dwTest : DWORD := 2#1000_0000_0000_0000_0000_0000_0000_0000;
END_VAR

dwTest := DRSTB(dwTest, 31);
(* dwTest = 2#0000_0000_0000_0000_0000_0000_0000_0000 *)
```

SRB, DSRB

Functions to Sets or reset Bit in a WORD or DWORD and return modified data.

SRB

Variable	Scope	Type	Description
IN	INPUT	WORD	WORD to change
iBN	INPUT	INT	What bit to reset 0-15
xState	INPUT	BOOL	Set if 1, reset if 0

Example:

```
VAR
    wTest : WORD := 2#1000_0000_0000_0000;
END_VAR

wTest := DSRB(wTest, 15, 0);
wTest := DSRB(wTest, 0, 1);
(* dwTest = 2#0000_0000_0000_0001 *)
```

DSRB

Variable	Scope	Type	Description
IN	INPUT	DWORD	DWORD to change
iBN	INPUT	INT	What bit to reset 0-31
xState	INPUT	BOOL	Set if 1, reset if 0

Example:

```
VAR
    dwTest : DWORD := 2#1000_0000_0000_0000_0000_0000_0000_0000;
END_VAR

dwTest := DSRB(dwTest, 31, 0);
dwTest := DSRB(dwTest, 0, 1);
(* dwTest = 2#0000_0000_0000_0000_0000_0000_0000_0001 *)
```

Regulators HYST, HYST_COOL

HYST (FB)

On\Off regulator function block for heating logic. It turns off when iPV become lower that iSV.

Variable	Scope	Type	Description
xIn	INPUT	BOOL	Enable regulator
iSV	INPUT	INT	Set value
iPV	INPUT	INT	Processed value
iDV	INPUT	INT	Delta
Q	OUTPUT	BOOL	ON or OFF

Here is an example how you can get a temperature on an AD0 and use it in hysteresis regulator to control heater on Y0 output.

Example:

```
fbHYST(
    xIn := xStart,
    iSV := 255, (* Set value is 25.5 *)
    iPV := AI_Temperature,
    iDV := 2, (* Delat is 0.2 *)
    Q := Y0
);
```

HYST_COOL (FB)

This is same Function Block it only logic in reverse. It turns off when `iPV` become bigger that `iSV`.

WORK_LEFT, WORK_LEFT_TIME

WORK_LEFT_TIME

Function to create a progress bar for a process to backward countdown from 100 to 0.

Variable	Scope	Type	Description
`ET`	INPUT	TIME	Elapsed time
`TW`	INPUT	TIME	Total time to work

Example:

```
fbTON(IN := xStart, PT := T#5m);
iTimeLeft := WORK_LEFT_TIME(fbTON.ET, fbTON.PT);
```

WORK_LEFT

Function to create a progress bar for a process to backward countdown from 100 to 0. It is a same timer but accept abstract units for time. Those might be seconds, or 100ms intervals like to `OUT_T` timers.

Variable	Scope	Type	Description
`ET`	INPUT	INT	Elapsed time
`TW`	INPUT	INT	Total time to work

Example:

```
OUT_T(xStart, TC0, 200);
iTimeLeft := WORK_LEFT(TN0, 200);
```

INCN SHIFT

INCN

There 2 functions were made for shitting primary array index.

Variable	Scope	Type	Description
`IN`	INPUT	BOOL	Make increment
`CUR`	INPUT	INT	Current number
`MAX`	INPUT	INT	Maximum number

Example:

```
iCount := INCN(TRUE, iCOunt, 2);
```

In this example `iCount` will rotate 0, 1, 2, 0, 1, 2, ... each PLC cycle.

SHIFT

This function shifts number within range

Variable	Scope	Type	Description
`SHIFTER`	INPUT	INT	For how many to shift
`IDX`	INPUT	INT	Current number
`MAX`	INPUT	INT	Maximum number

Example:

Main idea was this. Let's say you have an array of pumps or heaters and you start one by one as cascade. It means that pump in array index 0 will be always first and then next index and so on. What we want that from time to time we **shift** array index that we start.

Let's say we have array of 3 pumps.

```
iPumpsToStart := 2;
CASE iStep OF
10:
    MOV(xStart, 20, iStep);
    iShifter := INCN(iStep = 20, iCount, 2);

20:
    iStartedPumps := 0;
    FOR iCount := 0 TO 2 DO
        iShiftedCount := SHIFT(iShifter, iCount, 2);

        arPumps[iShiftedCount].xStart := iPumpsToStart > iStartedPumps;
        INC(arPumps[iShiftedCount].xStart, iStartedPumps);
    END_FOR;
END_CASE;
```

1. Every time we change from step 10 to step 20, `iShifter` will be incremented. When we do that first time `iShifter` = 1.
2. In step 20 `iCount` will rotate numbers 0, 1, 2 but `iShiftedCount` will rotate 1, 2, 0. It will start with array index 1 as first.
3. Next time we go from 10 to 20 `iShifter` = 2 and `iShiftedCount` will rotate 2, 0, 1. Now 3d pump will be main to start.

Helpers

L02_SET_IP

This function helps to initialize IP settings for L02 PLC.

Variable	Scope	Type	Description
`Init`	INPUT	BOOL	Command to set
`wts`	INPUT	INT	What type of IP address
`IP1`	INPUT	INT	First IP number
`IP2`	INPUT	INT	Second IP number
`IP3`	OUTPUT	INT	Third IP number
`IP4`	OUTPUT	INT	Forth IP number

Types of IP:

- `IP_PLC_IP` - IP address of a PLC
- `IP_PLC_GATEWAY` - PLC Gateway
- `IP_PLC_MASK` - PLC Mask
- `IP_REMOTE1` - Address of first remote EIP coupler
- `IP_REMOTE2` - Address of second remote EIP coupler
- `IP_REMOTE3` - Address of third remote EIP coupler
- `IP_REMOTE4` - Address of forth remote EIP coupler

Example:

```
M0 := L02_SET_IP(xSaveSettings, IP_PLC_IP, 192, 168, 0, 100);
M0 := L02_SET_IP(xSaveSettings, IP_PLC_GATEWAY, 192, 168, 0, 1);
M0 := L02_SET_IP(xSaveSettings, IP_PLC_MASK, 255, 255, 255, 0);
```

This is example how to setup network settings of L02 PLC ethernet port.

VALVE_3P

Function to control 3 position valve with PID. It is not and a pulse regulator but regulator with constant position search.

> Important !!! **This function required TimeControl library and TCO timer setup**

Variable	Scope	Type	Description
`ENABLE`	INPUT	BOOL	Start valve control

Variable	Scope	Type	Description
SV	INPUT	INT	Set valve position. It is 0-1000. Best configure PID task output to be 0-1000, If you created 0-100 output for PID, multiply it by 10.
TOTAL_TIME	INPUT	INT	Total time it takes for valve to move from fully CLOSED position to fully OPEN. Make it little Bit bigger (2%)
LUFT_TIME	INPUT	INT	Time required to move on direction change for valve to start moving.
DLT	INPUT	INT	Hysteresis for regulator to create non sense area. If difference between SV and current position is less that this value we do not move valve. It may reduce number of position changes when it is almost at the spot and save motor resources.
CLOSE_ON_DISABLE	INPUT	BOOL	When we turn off control with Enabled := FALSE should we close valve or leave it in a current position?
IS_OPENED	INPUT	BOOL	Feedback from valve
IS_CLOSED	INPUT	BOOL	Feedback from valve
OPEN	OUTPUT	BOOL	Open valve signal
CLOSE	OUTPUT	BOOL	Close valve signal

Example:

```
Valve_3p1(
    ENABLE := X0,
    SV := iPidTask,
    DLT := 50, (* 5.0% *)
    TOTAL_TIME := 10,
    LUFT_TIME := 500,
    CLOSE_ON_DISABLE := TRUE,
    IS_OPENED := X2,
    IS_CLOSED := X3,
    OPEN := Y0,
    CLOSE := Y1
);
```

SCALE, DSCALE, SCALE_NL

General functions scale value from one range to another range.

SCALE

Function to scale one value to another range proportionally and returns result as INT.

Variable	Scope	Type	Description
Val	INPUT	INT	Current value
inLow	INPUT	INT	Current value minimum
inHigh	INPUT	INT	Current value maximum
outLow	INPUT	INT	New value minimum
outHigh	OUTPUT	INT	New value maximum

For example you want to scale 0-100% of a PID regulator to analog output DA2 of an L02 host PLC.

Example:

```
D8052 := SCALE(iPIDTask, 0, 100, 0, 4000);
```

iPIDTask may have value from 0 to 100, and D8052 is a system register with control analog output DA2 which accepts values 0-4000.

DSCALE

Function to scale one value to another range proportionally and returns result as DINT.

Variable	Scope	Type	Description
Val	INPUT	DINT	Current value
inLow	INPUT	DINT	Current value minimum
inHigh	INPUT	DINT	Current value maximum
outLow	INPUT	DINT	New value minimum
outHigh	OUTPUT	DINT	New value maximum

Example:

```
diResult := SCALE(diVacuumSensor, 0, 32_000, 0, 100_000);
```

This is vacuum sensor from 0 Pa - 100 000 Pa.

SCALE_NL

Function to scale one value to another with none-linear proportions.

Variable	Scope	Type	Description
PN	INPUT	ANY16	Number of points

Variable	Scope	Type	Description
DTART	INPUT	ANY16	What device starts to store value
PV	INPUT	ANY16	Processed value on X scale to scale to Y

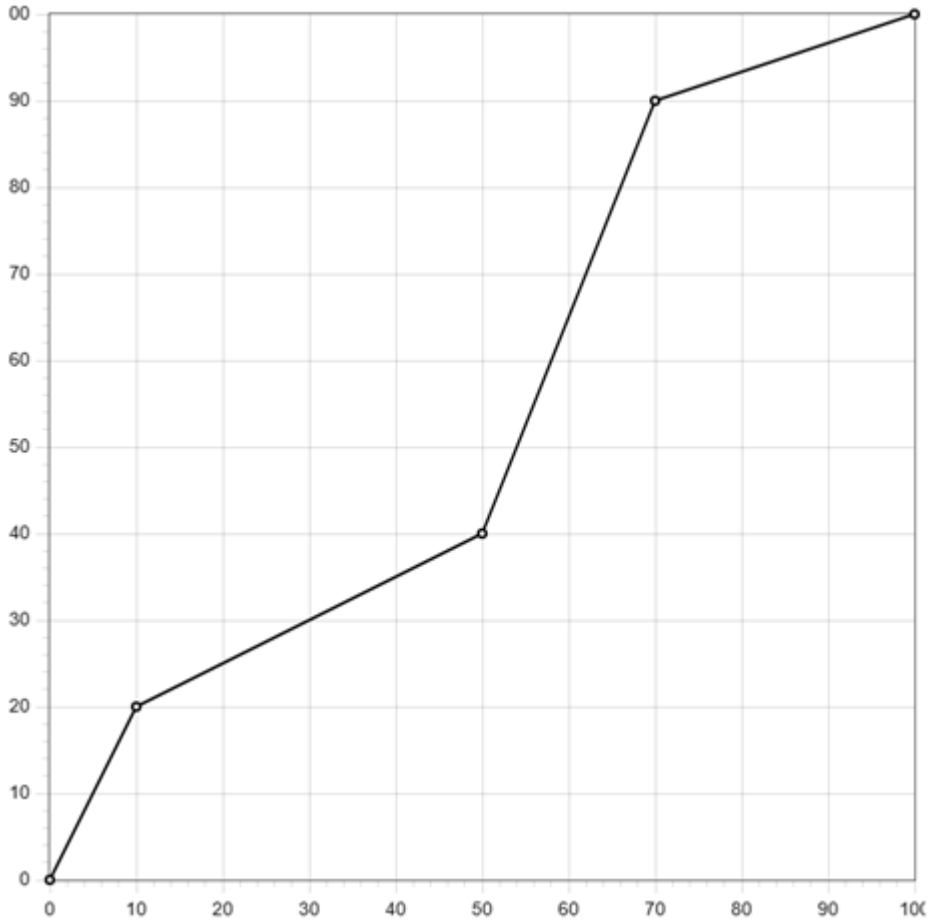
First you have to pack data. Let' say you want to create a 5 point graph started from D100. First device will keep number of points, and then points values.

```
D100 = Number of points
D101 = X1
D102 = X2
D103 = X3
D104 = X4
D105 = X5
D106 = Y1
D107 = Y2
D108 = Y3
D109 = Y4
D110 = Y5
```

Where X1-X5 and Y1-Y5 are not PLC inputs and outputs but point coordinates on X and Y scale. Let's say you have following values.

```
D100 = 5
D101 = 0
D102 = 10
D103 = 50
D104 = 70
D105 = 100
D106 = 0
D107 = 20
D108 = 40
D109 = 90
D110 = 100
```

This means we created a graph



Now we have 4 linear scale lines. Horizontal line is our measured value scale and vertical is what we convert it too. For instance is our `PV` will be 10 then output will be 20. If `PV` is 5 then output is 10. When you use Coolmay panel you can use XY Graph element, pass there D100 register and it will draw this graph.

In the code you can use it like this.

```
D200 := SACLE_NL(5, 100, Pv);
```

SCALE_AI, L02_SCALE_AI

Function to scale values from Analog inputs of PLC.

Sensor types

Scalable:

Following 3 types are scaled into measured units like Bar, Pa, cm, ... You set `Min` and `Max` for that.

- `STYPE_0_10V`
- `STYPE_0_20MA`
- `STYPE_4_20MA`

None-scalable:

This types are not scalable and returned as is, so `Min` and `Max` are not applied there. But it is still good practice to get values of those AI types through specialized function block as it initialize internal system Devices to set type of AI correctly.

- `STYPE_PT100`
- `STYPE_PT1000`
- `STYPE_TC_K`
- `STYPE_NTC`
- `STYPE_NTC10K`
- `STYPE_TC_E`
- `STYPE_TC_T`
- `STYPE_TC_S`
- `STYPE_TC_J`

SCALE_AI

Function to scale AI (Analog Input) of PLC host into measured units like Bar, Pa, cm, ... Traditionally Coolmay PLC occupy `D8030`-`D8045`, and it is 12bit input so range is from 0~4000.

Variable	Scope	Type	Description
`AINum`	INPUT	INT	Number of AI 0-16
`SType`	INPUT	INT	Type of the sensor. See list of types above.
`Min`	INPUT	INT	Minimum of measured unit. Only for 0-10V, 0-20mA or 4-20mA.
`Max`	INPUT	INT	Maximum of measured unit. Only for 0-10V, 0-20mA or 4-20mA.
`Correction`	INPUT	INT	Correction of output value.
`FilterTime`	INPUT	INT	Filter input by time. From 1ms to 60ms. Default 2ms.
`FilterNum`	INPUT	INT	Filtering cycles, default is 100 (range 2~20000), The larger value is, the result is more stable.
`ValueOut`	OUTPUT	INT	Scaled value
`ErrWire`	OUTPUT	Bit	Wire out error
`ErrLimit`	OUTPUT	Bit	Input values error. Minimum value is more than maximum.

This function block supports following types:

Example:

Let's say you have connected 4-20mA pressure sensor at AI0 (`AD0`). That sensor measure range is 0-16 bar. You want to convert values on that analog input to bars with precision of 0.1. And another sensor is AI1 (`AD1`) of PT100 type.

First declare function block.

```
VAR
    fbScale : SCALE_AI;
    AI0_Pressure: INT;
    AI1_Temperature: INT;
END_VAR
```

Then in a program

```
fbScale(
    AINum := 0,
    SType := STYPE_4_20MA,
    Min := 0,
    Max := 160,       (* 160 is equal to 16.0 *)
    FilterTime := 30, (* Increase filter for smoother result *)
    FilterNum := 200,
    ValueOut := AI0_Pressure
);

IF (fbScale.ErrWire) THEN
    (* Wire connection problem for a sensor AD0 *)
END_IF;

fbScale(
    AINum := 1,
    SType := STYPE_PT100,
    Correction := 5, (* Add +0.5 to value *)
    ValueOut := AI1_Temperature
);
```

Pay attention that we can use same function block to get values of all Analog Inputs and no need to create new instance for every AI.

L02_SCALE_AI

Function to scale AI (Analog Input) of L02 series PLC modules (L02-4AD, L02-RTD, ...) into measured units like Bar, Pa, cm, ... Values from modules are stored in `R23700`~`R23749` and are 16bit. It has value 0~32000.

Variable	Scope	Type	Description
`AINum`	INPUT	INT	Number of AI 0-49
`SType`	INPUT	ANY16	Type of the sensor. See list of types above
`Min`	INPUT	ANY16	Minimum of measured unit. Only for 0-10V, 0-20mA or 4-20mA.
`Max`	INPUT	ANY16	Maximum of measured unit. Only for 0-10V, 0-20mA or 4-20mA.
`Correction`	INPUT	ANY16	Correction of output value. Will be added to output
`ValueOut`	OUTPUT	ANY16	Scaled value

Example:

```
VAR
    fbScaleL02 : L02_SCALE_AI;
    AI0_Pressure: INT;
    AI1_Temperature: INT;
END_VAR
```

Then in a program

```
fbScaleL02(
    AINum := 0,
    SType := STYPE_4_20MA,
    Min := 0,
    Max := 160
    ValueOut := AI0_Pressure
);

fbScaleL02(
    AINum := 1,
    SType := STYPE_PT100,
    Correction := -2, (* Correction if -0.2 *)
    ValueOut := AI1_Temperature
);
```